

CSc 452 — Project 5: Virtual Memory Simulator

Report draft (export to PDF or DOCX for submission)

Add **your name** and **date** on the title page. This file includes **your** measured **gcc.trace** and **ls.trace** results.

1. Title page / header

- **Course & assignment:** CSc 452, Project 5 — Virtual Memory Simulator
 - **Your name / date**
-

2. Introduction

- **Purpose:** Compare four page-replacement policies (**OPT**, **CLOCK**, **Rand**, **NRU**) on realistic memory traces using a **single-level page table**, **16 KiB pages**, and a **parameterized frame count**.
- **Trace source:** Valgrind **lackey** traces (**gcc.trace**, **ls.trace**), decompressed from **.gz** as provided by the course.
- **Baseline:** **OPT** is the theoretical minimum page faults for each (*trace*, *frame count*) pair.

Paging matters because RAM holds only a subset of a process's pages; the replacement policy decides **which** resident page to discard on a fault and strongly affects **fault rate** and **I/O** due to dirty writebacks.

3. Implementation summary

Item	Your setting
Page size	16 KiB (16384 bytes)
Address mapping	page = address >> 14 (32-bit addresses)
Trace lines handled	I, L, S, M; invalid lines skipped
Modify (M)	One access; treated as read + write → sets dirty
NRU refresh	-r = 1000 memory references between clearing R bits
Quiet runs	-q suppresses per-access lines when collecting statistics

Algorithms simulated:

- **OPT:** Evict resident page whose **next use** is farthest in the future (efficient precomputation; not a naive full trace scan per fault).

- **CLOCK:** Second-chance clock with reference bit.
 - **Rand:** Random victim among resident frames (implementation-dependent seed).
 - **NRU:** Four classes from **(R, D)**; periodic **R** refresh every **1000** references.
-

4. Experimental methodology

1. **Primary workload:** `gcc.trace` — **1,711,763** memory accesses.
2. **Secondary workload:** `ls.trace` — **766,387** accesses (shorter trace; same algorithms and frame counts).
3. **Frame counts:** **8, 16, 32, 64**.
4. **NRU:** Single refresh interval `-r 1000` (documents “reasonable” choice per assignment; brief rationale below).

Commands used

```
./vmsim -q -n 8 -a opt gcc.trace
./vmsim -q -n 16 -a opt gcc.trace
./vmsim -q -n 32 -a opt gcc.trace
./vmsim -q -n 64 -a opt gcc.trace

./vmsim -q -n 8 -a clock gcc.trace
# ... repeat for clock, rand with -n 8 16 32 64

./vmsim -q -n 8 -a nru -r 1000 gcc.trace
# ... repeat -n 16 32 64

# Same pattern with ls.trace:
./vmsim -q -n 8 -a opt ls.trace
# ... etc.
```

5. Results — gcc.trace

5.1 Page faults (primary metric)

Frames	OPT	CLOCK	Rand	NRU (<i>-r 1000</i>)
8	11,357	20,455	27,469	44,116
16	2,581	6,068	7,643	7,544
32	428	844	1,265	1,254

Frames	OPT	CLOCK	Rand	NRU ($-r\ 1000$)
64	113	144	237	508

Observation: Faults drop sharply as frames increase. OPT always lowest at each column (by definition for this policy vs trace).

5.2 Fault ratio vs OPT (same frames)

Approximate **faults / OPT faults**:

Frames	CLOCK / OPT	Rand / OPT	NRU / OPT
8	1.80×	2.42×	3.89×
16	2.35×	2.96×	2.92×
32	1.97×	2.96×	2.93×
64	1.27×	2.10×	4.50×

(NRU at 64 frames underperforms OPT much more than CLOCK/Rand here — worth discussing: refresh period vs locality, class ordering, trace phase.)

5.3 Total writes to disk (dirty evictions)

Frames	OPT	CLOCK	Rand	NRU ($-r\ 1000$)
8	1,442	2,954	5,327	1,055
16	309	856	1,603	367
32	32	88	226	2
64	3	5	29	0

Observation: More faults generally correlate with more writebacks. NRU at **32/64** frames shows **very few** dirty evictions on this trace — different victim choice vs OPT/Rand/Clock.

5.4 NRU refresh choice ($-r = 1000$)

Rationale (short): A refresh every **1000** references periodically clears **R** bits so “not recently used” stays meaningful without freezing classification; **1000** is in a middle range — neither every few accesses (too aggressive) nor millions (stale R bits). You can add one sentence if you tried **500** or **2000** and saw similar fault counts.

5.5 Results — ls.trace (766,387 accesses)

Full runs — **not** comparable row-for-row with gcc (different program / locality); use for a **second graph** or appendix.

Page faults

Frames	OPT	CLOCK	Rand	NRU ($-r\ 1000$)
8	7,152	13,484	19,102	27,978
16	1,544	4,317	4,719	4,004
32	258	419	651	662
64	104	158	182	217

Fault ratio vs OPT (*ls.trace*)

Frames	CLOCK / OPT	Rand / OPT	NRU / OPT
8	1.89×	2.67×	3.91×
16	2.80×	3.06×	2.59×
32	1.62×	2.52×	2.57×
64	1.52×	1.75×	2.09×

On **ls.trace**, NRU at **64** frames is **not** an outlier like on **gcc.trace** — workloads differ.

Writes to disk (*ls.trace*)

Frames	OPT	CLOCK	Rand	NRU ($-r\ 1000$)
8	768	1,595	3,573	637
16	87	421	912	156
32	18	40	124	1
64	2	8	16	0

6. Graphs — *Page faults vs frames*

Charts exported from Google Sheets. NRU uses refresh $-r = 1000$ in both runs.

gcc.trace

(Many Markdown previews cannot render embedded PDFs; use PNG exports for inline previews, or paste these PDFs into your final submitted document.)

ls.trace

Page Faults vs. Frames (gcc.trace)

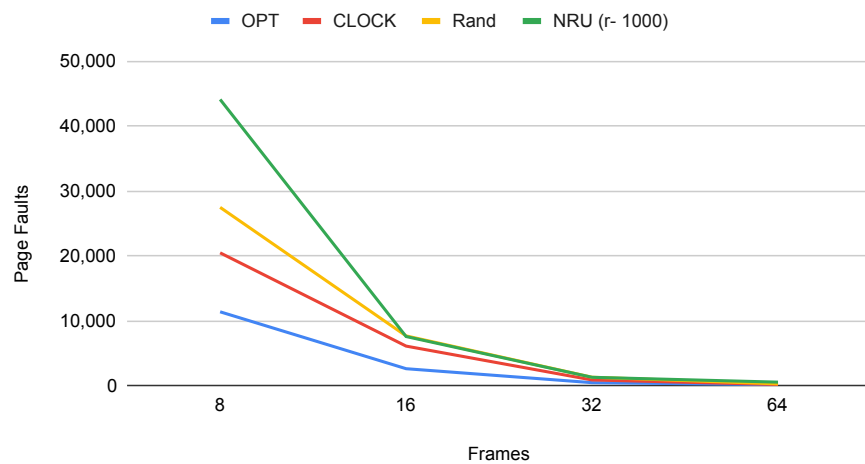


Figure 1: Page faults vs frames — gcc.trace

Page Faults vs Frames (ls.trace)

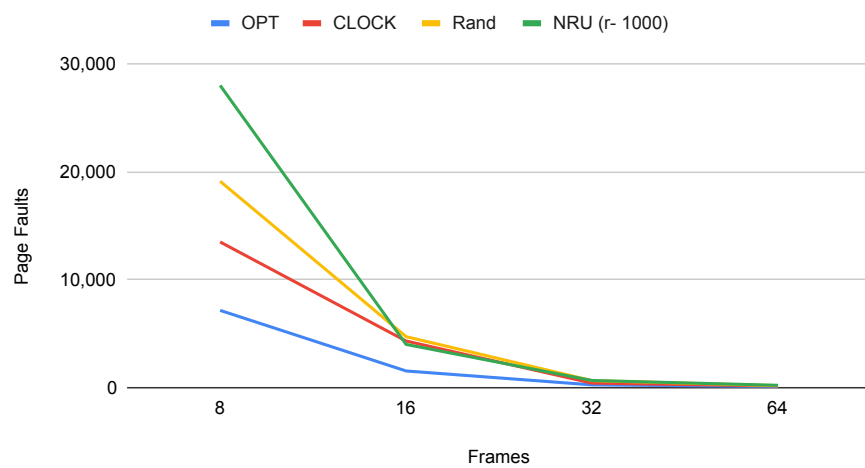


Figure 2: Page faults vs frames — ls.trace

7. Analysis & conclusions (*draft text — edit in your words*)

1. **vs OPT (gcc.trace):** CLOCK $\sim 1.3\text{--}2.4\times$ OPT faults; Rand $\sim 2.1\text{--}3\times$; NRU competitive at **8–32** frames but **much worse at 64** — discuss refresh + NRU class behavior vs this trace.
 2. **vs OPT (ls.trace):** Ratios differ (NRU at **64** is $\sim 2\times$ OPT here vs $\sim 4.5\times$ on gcc) — **same algorithm, different locality** — worth one sentence.
 3. **Scaling:** Doubling frames cuts faults sharply at low frame counts; diminishing returns toward **64** frames.
 4. **Rand:** Note fixed vs random seed; single run is a **sample** if nondeterministic.
 5. **Practical OS choice:** **OPT** unusable online. **CLOCK** — simple second-chance scan, strong in your tables. **Rand** — easy, unpredictable. **NRU** — needs **—r** tuning; behavior trace-dependent. Reasonable conclusion: **Clock** (approximate LRU class) for **cost vs performance**; cite **gcc** (and optionally **ls**) numbers.
-

8. References / honesty

- Traces from course-provided lackey outputs (**gcc.trace**).
 - **—q** used only to avoid printing $\sim 1.7\text{M}$ per-access lines when collecting summaries.
-

Before you submit

- ☐ Export to **PDF or DOCX**; add **graph**.
- ☐ Title page with **name**.
- ☐ **USERNAME-project5.tar.gz**: **source + report**, **not** trace files.
- ☐ **turnin** per course (**csc452-spring2026-p5** or current alias).